

PAUSTIK / PRODUCT AND DEVELOPMENT

Product and development specification

Neighborhood-first delivery from verified mother-led kitchens, with an operational plan for orders, money, and courier tracking.

Prepared for the Paustik pilot

This document describes the app as it exists in the current build and marks preview-only and planned capabilities clearly. Do not treat sample listings or values as live offers.

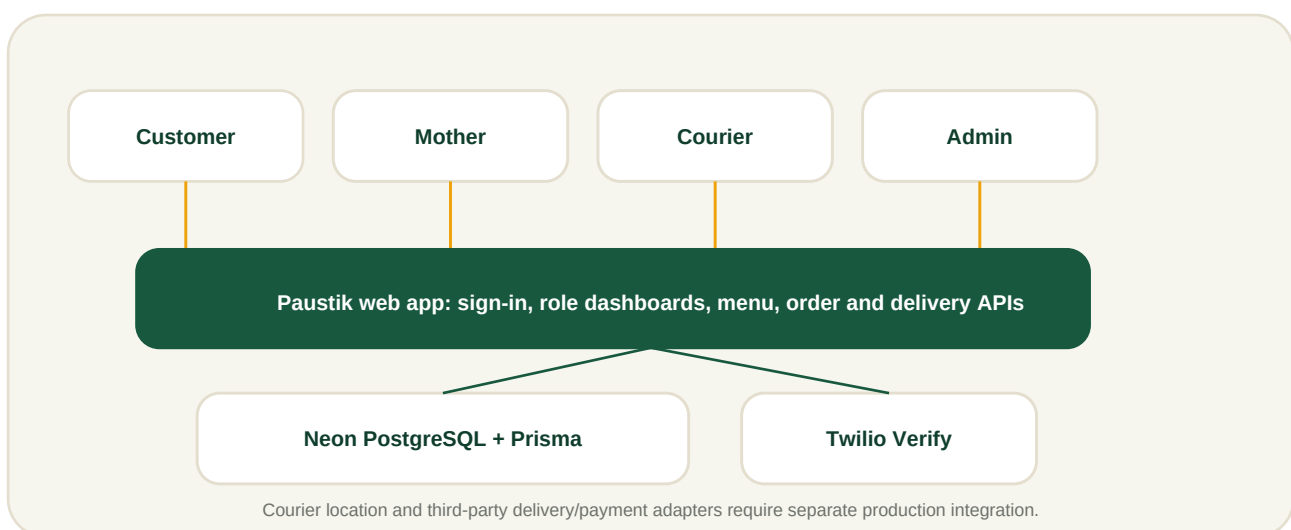
1. Product objective

Paustik connects local families with meals prepared by verified mother entrepreneurs. The pilot should prove one compact service area, predictable lunch menus, clear fees, safe kitchen onboarding and dependable handoff. It should not open broad delivery coverage until kitchen supply and courier capacity can meet each order window.

Current build status

Phone account sign-up, role routing, customer order-history queries, mother kitchen capacity capture, an Admin activity dashboard and a separate delivery-tracking pilot exist. Sample menus are a browser-only preview. Real checkout, settlement, marketplace dispatch, live OTP delivery and production cancellation terms require the integrations and policies described below.

2. Application architecture



The web app uses Next.js App Router with server-side pages and route handlers, Prisma for data access, Neon PostgreSQL when hosted, and a local SQLite fallback for local sign-in preview. Secrets stay on the server. Twilio Verify is the intended SMS/WhatsApp code provider. Location and payment providers should be integrated behind replaceable adapters.

Data and service boundaries

- Identity service: phone-based sign-in, role-scoped sessions, approval status and rate limits.
- Marketplace service: searchable verified kitchens, dated menu cycles, meal availability, stock, pricing and dietary categories.
- Order service: quote, checkout, payment state, immutable order events, cancellation/change requests, refunds and earnings ledger.
- Dispatch service: courier-provider adapter, assignment, accept/pick-up/drop-off events, ETA and customer tracking view.
- Operations console: kitchen approval, live order board, delivery exceptions, income/payout reconciliation, discounts and audit history.

3. Role access model

Role	Minimum access	Current state
Customer	Manage own profile and addresses; browse published kitchens; order, cancel within policy, track deliveries, review completed orders.	Account dashboard exists. Live discovery is database-backed; sample fallback is labelled. Checkout and marketplace order placement remain planned.
Mother entrepreneur	Manage own kitchen, cuisine, capacity, available days, lunch window, menus, stock, order acceptance and earnings.	Application captures kitchen profile. Menu-management tools and order acceptance need to be completed.
Delivery agent	See assigned jobs only; accept/reject within SLA; confirm pickup/dropoff; share location only for active delivery.	Registration and separate tracking pilot exist. Provider-backed marketplace assignment remains planned.
Admin	Approve accounts and kitchens; manage catalog, delivery zones, policies, voucher rules and finance; review audit activity.	Protected admin dashboard and admin account management exist. Extend with role-based scopes and production data reconciliation.
Partner	Model as Mother/Kitchen Partner for the pilot; separate label can be added without extra authority.	No distinct database enum value; mother is the existing partner workflow.
Developer	Read-only diagnostics, test data and technical status; no payouts, customer PII export or role escalation by default.	Not present in the role enum. Add only through a reviewed migration and explicit least-privilege guards.

4. Marketplace and menu requirements

- Find kitchens by verified status, accepting-orders flag, customer PIN/locality and service radius. Use distance and explicit neighbourhood labels; do not call a kitchen nearby without a usable address or coordinate.
- Kitchen profile: mother name, kitchen name, cuisine specialties, hygiene verification status, service days, lunch window, daily capacity, rating count and average rating.
- Menu cycle: draft, submitted, approved, published, retired. Each dated entry records meal period, vegetarian/non-vegetarian/vegan category, price, stock, ingredients and allergens.
- Mother availability: prevent accepting more meals than remaining daily capacity. Give the mother a pause control and an explicit substitute/decline workflow.
- Rating eligibility: accept a review only from a completed order. Store meal and delivery scores separately and prevent duplicate review per order and subject.

5. Order, cancellation and meal-change rules

Recommended lifecycle: Cart -> Quoted -> Payment pending -> Placed -> Accepted by kitchen -> Preparing -> Ready for pickup -> Courier assigned -> Picked up -> En route -> Delivered. Exception outcomes include Rejected, Cancelled, Payment failed, Delivery failed, Refunded and Partially refunded. Record every transition as an append-only order event with actor, timestamp and reason.

Policy decision	Required before launch
Customer cancellation	Set a clear cutoff before kitchen preparation; define full/partial refund based on preparation state and local consumer rules.
Meal change	Allow only before the published cutoff and only when the replacement is available and price difference is handled.
Kitchen rejection	Release payment authorization or initiate refund, notify customer and update courier assignment.
Late or failed delivery	Set contact path, evidence requirements, refund/credit authority and responsibility split.
Subscription pause/skip	Set notice cutoff, skipped-day billing behavior and renewal date rules before activating plans.
Do not invent policy terms in code The current app has cancellation and meal-change data models, but Paustik has not supplied final cutoffs, refund rules or subscription terms. Keep these rules configurable and publish them before taking payment.	

6. Payment and income accounting

Show customers a line-item quote before payment and store each financial amount in INR with a payment provider reference and currency. Recommended order components:

Customer charge	Operations ledger / settlement
Meal subtotal	Mother gross earnings
Delivery fee	Courier payable or third-party courier pass-through
Tax	Tax collected/payable and invoice reference
Platform fee	Paustik revenue / commission
Voucher discount	Promo liability and funding party
Refund or adjustment	Refund amount, reason and related payment event

Admin income view should separate GMV, Paustik fee revenue, mother earnings, courier cost, collected tax, voucher discounts, refunds, unsettled funds, pending payouts and completed payouts. Never calculate income from order totals alone. Reconcile payment-provider webhooks idempotently before showing paid or settled status.

7. Delivery tracking and geofencing

- Begin with one configured pilot zone and service radii. Show customer-entered address and PIN code; use geolocation only after explicit consent.
- For third-party delivery, create an adapter interface for quote, create task, cancel task, webhook and tracking URL. Save provider and external job ID; validate signatures and make event handling idempotent.
- Courier web tracking is foreground-only. Ask for location permission after assignment; send timestamped latitude, longitude and accuracy during an active trip. Do not request always-on background access in the first web pilot.
- Server-side geofence checks use distance-to-pickup/dropoff, reported accuracy, fix age and dwell time. A stale or low-accuracy GPS fix cannot alone mark pickup or delivery complete.
- Use a pickup radius and destination radius as operational configuration, not hard-coded trust. Let the courier and customer confirm handoff with an OTP or evidence path.
- Retain only the location points needed for an active delivery and incident window. Stop capture at completion/cancel; restrict operations access and log reads.

8. Promotions and vouchers

Support code, start/end, eligible zone, eligible customer cohort, first-order flag, minimum basket, percentage/fixed discount, maximum discount, use count, per-customer limit, funding owner and redemption state. Validate and redeem in one transaction to prevent concurrent overuse. Show discount and conditions before payment.

9. Database operations and free tooling

Tool	Use	Guardrail
Neon Console Tables	Quick visual inspection and record edits for the current Neon database.	Use a development branch for edits; production record changes are immediate.
Prisma Studio	Project-aware local GUI for PostgreSQL or local SQLite records.	Launch only after setting the correct DATABASE_URL; use read-only access for audit.
DBaver Community	Free desktop PostgreSQL client with table and query editors.	Use SSL and a restricted database role; do not save production credentials in shared folders.
Neon MCP agent	Natural-language database setup and inspection for Codex/Cursor and other agents.	Not connected in this Codex session. Neon says development/testing only; review each agent action.

10. Environment, security and deployment

- Required hosted foundation: DATABASE_URL, AUTH_SECRET and Twilio Verify key/service identifiers for actual SMS/WhatsApp. Keep all server-side; never expose with NEXT_PUBLIC_.
- Demo auth displays codes in the browser and bypasses phone ownership proof. Keep disabled for a real public launch. Admin registration stays protected behind existing admin access or a separately controlled first-admin bootstrap.
- Use one migration per schema change. Review SQL for data loss, run against a Neon development branch, validate the release build, then apply additive production migrations before deploying code that depends on them.
- Protect role APIs on the server, not only in navigation. Audit administrator writes. Limit OTP requests and failed code checks; use secure httpOnly session cookies.
- Deploy through Vercel after checks. Verify sign-up, login, role redirect, DB writes and production logs after release. Do not call the marketplace production-ready while payment/courier integrations are stubs.

11. Delivery roadmap

Phase	Deliverables	Exit check
A - Account and profiles	SMS/WhatsApp configuration, profile fields, safe first-admin setup, admin role management and approvals.	Create each role, sign in, verify stored profile and enforce permissions.
B - Neighborhood catalog	Verified kitchens, menu editor, weekly cycles, dietary categories, stock/capacity and locality search.	Customer sees only approved, available listings with correct prices.
C - Order and money	Cart, quote, payment provider, cancellation/change configuration, vouchers, refunds, mother earnings and reconciliation.	Payment webhook creates one order/ledger entry; refunds reconcile correctly.
D - Dispatch and tracking	Courier adapter, assignment, tracking, geofence and exceptions.	Simulate accept/pickup/dropoff; stale GPS and duplicate webhooks are rejected safely.
E - Pilot operations	Support workflow, food-safety evidence, reports, monitoring and limited service area.	Operate a measured pilot with response times, on-time delivery, quality and payout reconciliation.

12. Minimum verification matrix

Flow	Expected verification
Sign-up	SMS and WhatsApp channel selection; invalid phone/code; duplicate phone/email; mother fields persisted; safe redirect.
Role access	Customer cannot open admin pages; pending mother/courier cannot receive live work; every admin write is recorded.
Menu	Draft is invisible to customer; published meal within date/stock/capacity is visible; veg/non-veg and price are accurate.
Checkout	Totals add up; voucher limits are enforced; payment duplicate webhook does not duplicate an order.
Delivery	Courier consent, location update, stale/low-accuracy point, pickup/dropoff geofence, cancellation and handoff OTP.
Admin income	Order charge, platform fee, mother earnings, courier cost, voucher, refund and payout reconcile to source events.

References

Twilio Verify: [verification API](#) and [WhatsApp channel](#). Database tools: [Neon Tables](#), [Prisma Studio](#), [DBever Community](#), and [Neon MCP security guidance](#).